

Local LLMs

-

(presque)

ChatGPT, gratuit* et sur votre machine!

Un LLM, c'est quoi ?

LLM = Large Language Model

C'est :

- Un réseau de neurones dérivé de l'architecture transformer
- Un “tokenizer”
- Un “contexte”
- Des “poids”

Un tokenizer ? Il lit pas mes mots directement ?

Un LLM n'opère que sur des “token”, des symboles représentant 1 ou plusieurs syllabes, mots, ou caractères spéciaux.

En moyenne, 100 tokens correspondent à 75 mots [1].

L'input et l'output de tout LLM est exclusivement composé de **tokens**, pas de mots, de lettres ou autre forme de données.

Chaque LLM a un “vocabulaire” de tokens différent.

Visualisation : <https://platform.openai.com/tokenizer>

Le contexte : la “mémoire” de votre LLM

Les tokens composant l'input du LLM s'appellent le **contexte** , il est :

- Limité : le contexte ne peut pas être infini*
- Déterminant pour la rapidité d'exécution : l'algorithme de “self-attention” est en $O(n^2)$ pour la plupart des architectures de LLM modernes
- Important à conserver : Les performances des LLM diminuent plus leur contexte est chargé [2]

Les poids : le réseau lui-même

Les poids sont le gros des données dans un fichier contenant un LLM, ce sont les paramètres du réseau de neurone correspondant, en général :

- + de poids “activés” => de meilleures performances, mais un modèle plus lent à l'exécution
- - de poids “activés” => plus de rapidité d'exécution, mais de moins bonnes performances

Les poids : “activés”, ça veut dire quoi ?

Les modèles se partagent entre 2 grandes catégories : “dense” et “mixture of experts” (MoE) :

- les “dense” utilisent 100% de leurs paramètres pour chaque token généré
- les MoE n’en utilisent qu’une petite partie, donc ils ont moins de poids “activés” pour chaque token

À quantités de poids total égales, un modèle dense est généralement plus performant, même si il est plus lent !

**Maintenant, comment faire tourner ça sur
nos machines ?**

Déjà, pourquoi faire tout ça en local ?

Les 2 raisons principales sont les suivantes :

- La sécurité de vos données : il est impossible actuellement de faire de l'inférence E2EE, le serveur aura **toujours** une version complètement déchiffrée de votre prompt à un moment donné, en local, vous n'avez pas besoin de faire confiance à un serveur tiers
- Le contrôle : en local vous pouvez faire tourner les modèles que vous voulez du moment que vous avez le logiciel pour et le fichier du modèle, alors que les providers cloud changent fréquemment leurs offres, que ce soit les modèles, leur paramétrage ou leurs prix

Est-ce qu'il me faut un PC de la NASA ?

Non ! Pas besoin d'un supercalculateur pour faire tourner des LLM en local, pour apprendre à s'en servir, cela suffit largement :

- Stockage : ~6 à 10 Go
- RAM : ~5 Go libre
- Calcul : un CPU
- Réseau : juste pour installer le logiciel et télécharger le modèle

Après, pour faire tourner des plus gros modèles, il va falloir plus de RAM, de stockage, et potentiellement un GPU pour que ce soit plus rapide, mais c'est bien optionnel !

Le moteur d'inférence, le nerf de la guerre

L'inférence, c'est la génération du prochain token le plus probable à partir du contexte et des poids du modèle. Le logiciel qui effectue ce calcul s'appelle le “moteur d'inférence”, Il y en a plusieurs choix :

Nom	llama.cpp	vllm	MLX
Desc	- Fonctionne sur tout - Setup trivial partout - pas le plus optimisé	- meilleures performances sur GPU et multi-GPU (surtout Nvidia) - Setup complexe	- meilleures performances sur Apple Silicon - Setup simple

L'interface graphique

Si vous voulez avoir une interface graphique toute faite pour interagir avec votre LLM local et/ou choisir vos modèles, il y a plusieurs options aussi :

Nom	Ollama	KoboldCpp	OpenWebUI	Jan	LM Studio
Inférence	llama.cpp*	llama.cpp	BYO	llama.cpp/BYO	llama.cpp
Open source	oui	oui	oui	oui	non
Remarques	- un peu plus lent - facile à setup - détails cachés	- facile à setup - moins de fonctionnalités	- le + proche de ChatGPT	- fonctionnalités avancées	- facile à setup

Le choix du modèle

Vous êtes équipés d'un moteur d'inférence et d'une interface graphique, tout ce qu'il vous manque est un modèle !... Comment le choisir ?

VL = Vision Language (peut tokeniser des images)

3B paramètres/poids actifs
(seulement pour les MoE)

Qwen3-VL-30B-A3B-Instruct

Série de modèle ou auteurs du modèle

Nombre de paramètres total, pour les modèles denses, ce sont aussi les paramètres actifs

Variante faite pour le chat, les "Reasoning" ou "Thinking" sont mieux pour les sciences, les "Base" sont à éviter pour ces usages !

Le choix du modèle : d'autres noms

Il n'y a pas de convention universelle sur le nommage de modèles open-weights, voici quelques exemples en plus :

- GPT-OSS-20B : modèle MoE avec 3,6B de poids actifs de la part d'OpenAI
- Llama-4-Scout-17B-16E-Instruct : modèle MoE avec 17B poids actifs et 16 experts de la part de Meta, peut traiter les images et a un total de poids de ~109B poids
- DeepSeek-V3.2 : modèle MoE avec ~37B poids actifs et ~671B poids total de la part de DeepSeek

Si un nom de modèle n'est pas assez clair et que votre interface ne vous fournit pas assez d'informations, n'hésitez pas à vous renseigner sur la page Hugging Face des modèles précis, par exemple : <https://huggingface.co/openai/gpt-oss-20b>

Le choix du modèle : la quantization

Pour économiser de la place sur disque et de la place en mémoire à l'exécution, il est possible d'utiliser des modèles “quantisés”.

ATTENTION : la quantization est un processus de compression **A PERTES**, le modèle résultant n'est **jamais** aussi performant que la version originale !

Cela dit, la quantification pour les modèles au format GGUF (celui de llama.cpp) est généralement notée de cette manière :

Ministral-3-14B-Instruct-2512-Q4_K_M.gguf

Chaque poids fait ~4 bits

Les activations sont “moyennement” quantisées

Les quantizations < Q4 sont généralement de trop mauvaise qualité pour la plupart des utilisateurs !

Les paramètres d'exécution

On y est presque ! Vous avez l'inférence, l'interface, le modèle, il ne reste plus qu'à l'exécuter... mais il faut bien choisir comment ! Votre interface vous laissera probablement choisir les paramètres suivants :

- La taille du contexte en tokens : cela va influencer la RAM/VRAM que le contexte va occuper, trop petit et vos prompt dépasseront le contexte, trop grand et le modèle deviendra incohérent et l'exécution ralentira, en général les modèles locaux modernes sont faits pour une taille de contexte de 8 à 32k, des fois 64k+ (info souvent sur Hugging Face !)
- La longueur maximale de la réponse en tokens : certains modèles “raisonneurs” peuvent avoir besoins de milliers pour produire une réponse, d'autres sont faits pour répondre en quelques centaines, en fonction de vos instruction les réponses peuvent aussi être plus ou moins longues, ~2 à 4k est souvent plus qu'assez pour les “Instruct”

Les paramètres d'exécution (partie 2)

- La température : contrôle (en partie) le taux d'aléatoire injecté dans les réponses du modèle, en général à faire varier entre 0 ("déterministe") et 1 ("créatif"), certains modèles sont entraînés pour une température en particulier (ex: 0.7 pour les Qwen3)
- Le nombre de couches du modèle sur GPU : si vous n'êtes pas stressés sur l'optimisation, laissez le en automatique, sinon allez chercher des guides en lignes parce que si le auto est pas 100% ça devient compliqué (mais ça vaut le coup pour la vitesse supplémentaire !)

Et maintenant, la pratique !

Vous avez la théorie, passez à la pratique, essayez :

- Interface graphique : Koboldcpp
(<https://github.com/LostRuins/koboldcpp/releases/tag/v1.103>)
- Modèle : <https://huggingface.co/mistralai/Mistral-3-3B-Instruct-2512-GGUF>
- Température : 0.7
- Context window : 16k

Appelez nous si vous avez besoin d'aide

**Ok, et comment est-ce qu'on utilise ça
dans un programme ?**

L'API OpenAI standard

OpenAI ont été les premiers à fournir une API REST populaire pour leurs LLMs, et tous les moteurs d'inférences et interfaces l'ont copié pour fournir une API locale, et ça s'utilise parfaitement bien en python !

L'API OpenAI standard

```
import openai

openai.base_url = "http://localhost:5001"
openai.api_key = 'sk-WHOCARES'

completion = openai.chat.completions.create(
    model="whatever.gguf",
    messages=[
        {
            "role": "system",
            "content": "Réponds en tant que capitaine d'un bateau pirate" ,
        },
        {
            "role": "user",
            "content": "Comment est-ce que je peux faire un backflip ?" ,
        },
    ],
)

print(completion.choices[0].message.content)
```

**Et pourquoi tout le monde fait pas de l'IA
locale ?**

Les limites de l'IA locale : la qualité

- Les meilleurs modèles fournis par les service cloud sont souvent de très grande taille par rapport à ce que vous pouvez faire tourner sur vos machines, par exemple GPT-4 (2023) : MoE avec ~1800B paramètres au total.
- Les labos d'IA fermés gardent certaines astuces secrètes, par exemple la gestion des très longs contextes (ex : Gemini ≥ 2.5)
- Les modèles locaux comparables en performance aux meilleurs modèles gratuits illimités sont autour de 20-40B

Les limites de l'IA locale : la vitesse

- Pour exécuter un gros modèle d'IA rapidement, il **faut** tous les poids en **VRAM**, sur un GPU
- Même avec 64Go de RAM DDR5, vous n'exécuterez jamais un modèle dense $\geq 32B$ plus rapidement que 1-2 tokens par seconde
- Un modèle conversationnel a généralement besoin de 7-12 tokens par seconde pour ne pas avoir l'air "lent" pour la plupart des utilisateurs
- Un modèle "raisonneur" a généralement besoin de ≥ 30 tokens par seconde

Les limites de l'IA locale : les outils

Aujourd'hui, une interface d'IA se limite rarement à l'inférence pure, on a souvent des outils supplémentaires : recherche internet, base de données, exécution de python, calcul mathématique...

Il y a des alternatives à ces outils en local, mais :

- Le setup est pas toujours facile
- Certains de ces outils dépendent de choses pas facilement self-hostables (ex : recherche internet)
- Les standards évoluent et changent en permanence, compliqué de garder la cadence seul !

Les limites de l'IA locale : le prix

Hein ? mais c'est pas gratuit ?

Si vous l'utilisez sur du hardware que vous avez pris pour autre chose, avec de l'électricité que vous payez pas (e.g. ordi perso sur le campus) alors oui, mais...

- Les CPU et même GPU consommateurs sont bien moins efficaces pour ce type de travail que les setup de datacenter
- Si vous voulez spec un serveur d'inférence personnel, il est presque certain que dépenser le même argent dans des services de location de hardware adapté ou d'inférence (ex : runpod, openrouter) vous apportera une bien meilleure performance (sans la sécurité des données)

Pour aller plus loin...

Pour aller plus loin

- Les modèles fine tuned par la communauté
- Le déploiement multi-utilisateurs
- Optimiser ses paramètres d'inférence (ex : `--n-cpu-moe` sur llama.cpp)
- Arriver à faire marcher tout ça sur un GPU AMD sous ROCm (bonne chance)
- Systèmes de STT (Speech to text) et TTS (Text to speech)
- Intégrer un LLM local à votre IDE (ex : Continue sous VSCode)
- Le fine tuning (il vous **faut** un bon GPU)

Bibliographie

[1] <https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>

[2] <https://longbench2.github.io/>

Crédits

Slides : Pierre Weisse